

การออกแบบบริการ (Service Design): ฟีเจอร์ Team Chat สำหรับระบบ Task Board

1. บทนำ (Introduction)

เอกสารฉบับนี้นำเสนอการออกแบบฟีเจอร์ใหม่ชื่อว่า Team Chat ซึ่งถูกพัฒนาเพิ่มเติมให้กับระบบ Task Board โดยใช้แนวคิดสถาปัตยกรรมแบบ Microservices

ฟีเจอร์ที่พัฒนาประกอบด้วย:

- ระบบแชทแบบ Real-time ภายในแต่ละ Board
- การแจ้งเตือนเมื่อมีข้อความใหม่
- การค้นหาประวัติการสนทนา (Chat History)
- รองรับการส่งไฟล์ (File Sharing)

เป้าหมายของการออกแบบนี้คือการสร้างระบบที่สามารถ รองรับการขยายตัว (Scalable), ดูแลรักษาง่าย (Maintainable) และมี ประสิทธิภาพสูง (High Performance)

2. การตัดสินใจออกแบบ Service

2.1 ควรแยก Chat Service หรือรวมกับ Task Service?

ในระบบนี้เลือกออกแบบให้ Chat Service เป็น Microservice แยกออกมา จาก Task Service เหตุผล

1. Separation of Concerns (แยกหน้าที่การทำงาน)
 - Task Service ดูแลการจัดการงาน (CRUD)
 - Chat Service ดูแลการสื่อสารแบบ Real-time
2. ความต้องการด้านการ Scale แตกต่างกัน
 - ระบบ Chat ต้องรองรับข้อความจำนวนมากและแบบ Real-time
 - ระบบ Task มีปริมาณการใช้งานน้อยกว่า
3. ความง่ายในการดูแลรักษา (Maintainability)
 - สามารถ Deploy และ Update แยกกันได้
 - ลดความซับซ้อนและการพึ่งพากันของระบบ
4. การเพิ่มประสิทธิภาพ (Performance Optimization)
 - สามารถปรับแต่ง Chat Service ให้เหมาะกับงาน Real-time ได้โดยเฉพาะ

3. เทคโนโลยีที่เลือกใช้ (Technology Stack)

3.1 Backend Framework

- Node.js (NestJS หรือ Express)
- เหตุผล: รองรับการทำงานแบบ Non-blocking และ Event-driven เหมาะกับระบบ Real-time

3.2 การสื่อสารแบบ Real-time

- WebSocket (Socket.IO)
- เหตุผล: มีความหน่วงต่ำ (Low latency) และสื่อสารได้สองทาง (Full-duplex)

3.3 Message Broker

- Apache Kafka หรือ RabbitMQ
- เหตุผล:
 - รองรับ Event-driven Architecture
 - ลดการพึ่งพาระหว่าง Service
 - มีความน่าเชื่อถือในการส่งข้อมูล

3.4 ระบบจัดเก็บไฟล์

- Amazon S3 หรือ MinIO
- เหตุผล:
 - รองรับการจัดเก็บไฟล์ขนาดใหญ่
 - สามารถขยายระบบได้ง่าย

4. การออกแบบฐานข้อมูล (Database Design)

4.1 ฐานข้อมูลหลัก: MongoDB (NoSQL) เหตุผล

- โครงสร้างข้อมูลยืดหยุ่น (Flexible Schema)
- รองรับการจัดเก็บข้อมูลจำนวนมาก (High Write Throughput)
- สามารถขยายระบบแบบ Horizontal ได้ง่าย

ตัวอย่างข้อมูล:

```
{
  "_id": "messageld",
  "boardId": "b1",
  "userId": "u1",
  "content": "Hello",
  "type": "text",
  "timestamp": "2026-03-22T10:00:00Z"
}
```

4.2 ระบบ Cache: Redis

ใช้สำหรับ:

- เก็บข้อความล่าสุด (Recent Messages)
- จัดการสถานะผู้ใช้งาน (Online Users)
- ใช้ Pub/Sub เพื่อรองรับ WebSocket หลาย Server

4.3 ระบบค้นหา (Optional): Elasticsearch

ใช้สำหรับ:

- ค้นหาข้อความแบบ Full-text
- ดึงข้อมูล Chat History ได้รวดเร็ว

5. การออกแบบระบบ Real-time

5.1 เปรียบเทียบ WebSocket กับ Polling

	หัวข้อ	WebSocket	Polling
Latency		ต่ำ	สูง
ประสิทธิภาพ		สูง	ต่ำ
ภาระ Server		ต่ำ	สูง
Real-time		ดีมาก	ไม่เหมาะ

สรุป: เลือกใช้ WebSocket เนื่องจากสามารถสื่อสารแบบ Real-time ได้อย่างมีประสิทธิภาพ และเหมาะสมกับระบบ Chat มากที่สุด

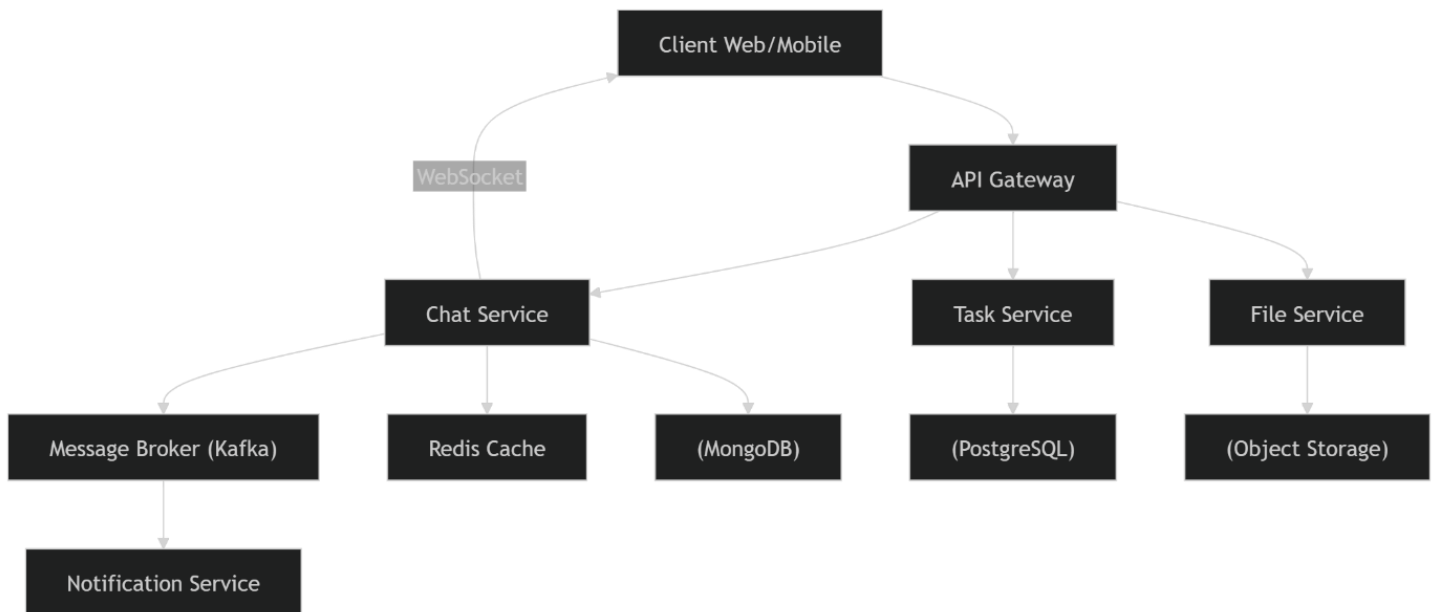
6. ภาพรวมสถาปัตยกรรมระบบ (System Architecture Overview)

องค์ประกอบหลัก:

- Client (Web / Mobile)
- API Gateway
- Task Service
- Chat Service
- Notification Service
- File Service
- Message Broker (Kafka)
- Database (MongoDB, Redis, PostgreSQL)

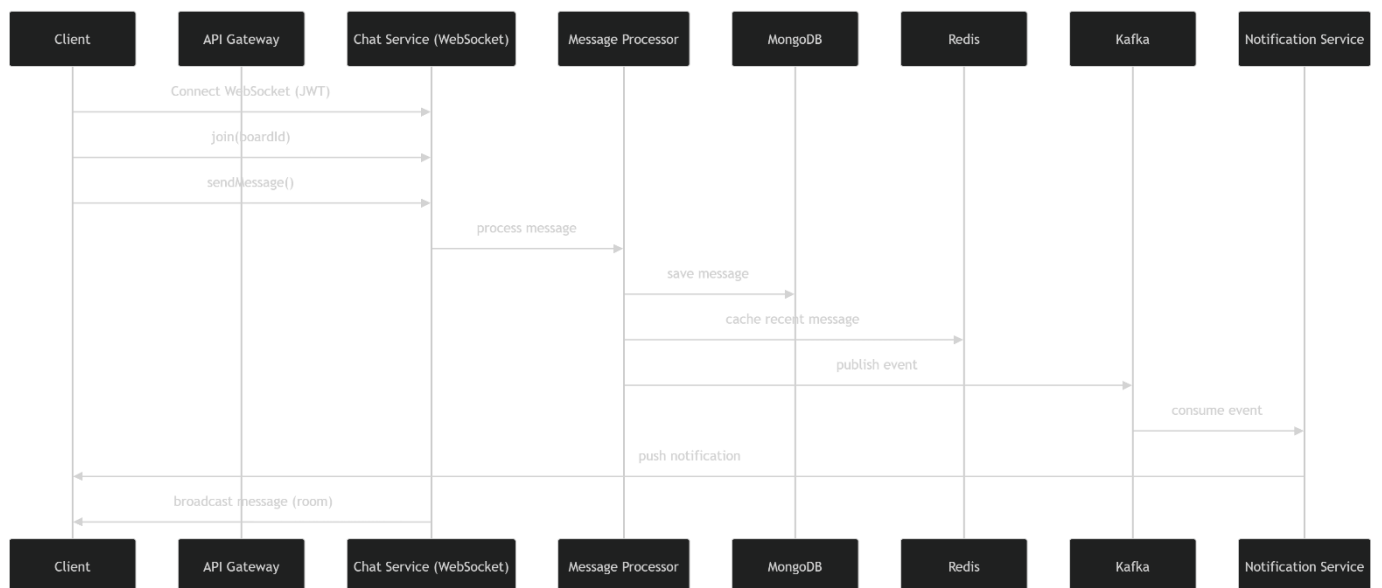
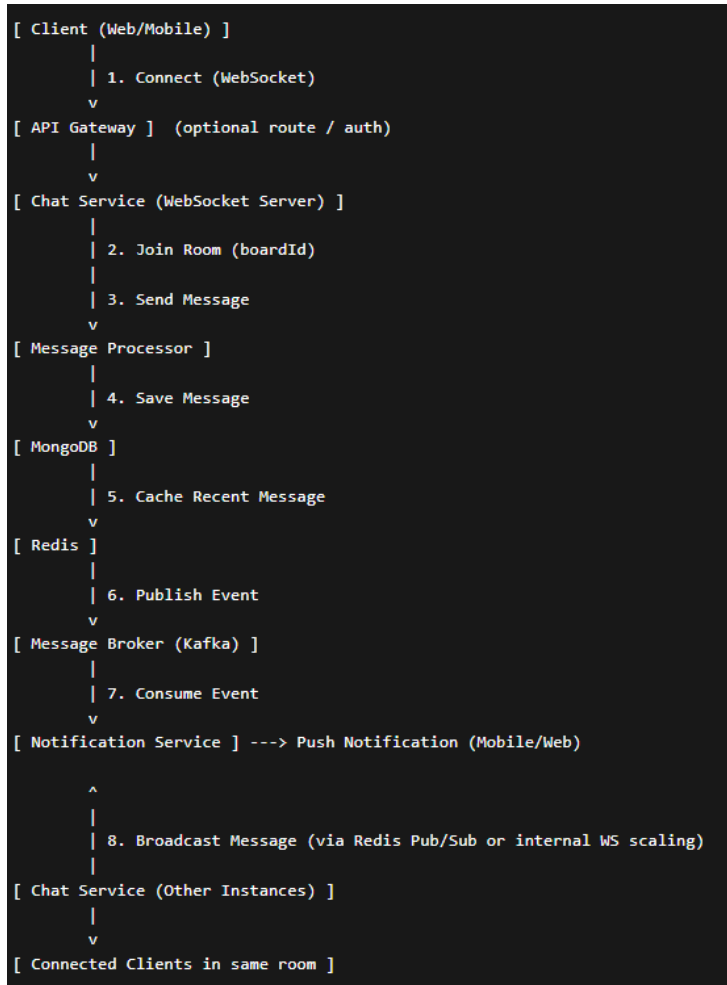
องค์ประกอบภายใน Chat Service:

- WebSocket Server
- REST API Controller
- Message Processor
- Event Publisher
- Database Layer



7. การทำงานของ WebSocket (WebSocket Communication Flow)

1. Client เชื่อมต่อกับ WebSocket Server
2. Client เข้าร่วมห้องแชท (boardId)
3. Client ส่งข้อความ
4. Chat Service ประมวลผลและบันทึกลงฐานข้อมูล
5. ส่ง Event ไปยัง Message Broker
6. กระจายข้อความไปยังผู้ใช้ในห้องเดียวกัน



8. สรุป (Conclusion)

การออกแบบ Chat Service ในรูปแบบ Microservices นี้มีข้อดีดังนี้:

- รองรับการขยายระบบ (Scalability)
- มีประสิทธิภาพสูงด้วย WebSocket และ Cache
- มีความน่าเชื่อถือด้วย Event-driven Architecture
- มีความยืดหยุ่นด้วย NoSQL Database

สถาปัตยกรรมนี้สามารถรองรับผู้ใช้งานจำนวนมาก และเหมาะสมกับระบบ Chat แบบ Real-time อย่างมีประสิทธิภาพ